Figure 34.4 The Quartz.NET scheduler starts on app startup and logs its configuration. The default configuration stores the list of jobs and their schedules in memory and runs in a nonclustered state. In this example, you can see that no jobs or triggers have been registered, so the scheduler has nothing to schedule yet.

TIP Running your application before you've added any jobs is good practice. It lets you check that you have installed and configured Quartz.NET correctly before you get to more advanced configuration.

A job scheduler without any jobs to schedule isn't a lot of use, so in the next section we'll create a job and add a trigger for it to run on a timer.

34.3.2 Configuring a job to run on a schedule with Quartz.NET

In section 34.1 we created an `IHostedService` that downloads exchange rates from a remote service and saves the results to a database using EF Core. In this section you'll see how you can create a similar Quartz.NET `IJob` and configure it to run on a schedule.

The following listing shows an implementation of `IJob` that downloads the latest exchange rates from a remote API using a typed client, `ExchangeRatesClient`. The results are then saved using an EF Core `DbContext`, `AppDbContext`.

Listing 34.11 A Quartz.NET `IJob` for downloading and saving exchange rates

```
public class UpdateExchangeRatesJob : IJob                          ❶
{
    private readonly ILogger<UpdateExchangeRatesJob> _logger;       ❷
```